

Exploring shapely

```
In [... import shapely

p = shapely.Point(1,2)
```

```
In [... p
```

```
Out [...
```



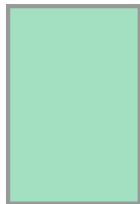
That isn't a placeholder, the point is being displayed on the screen.

```
In [... poly = shapely.Polygon([(0,0),(2,0),(2,3),(0,3),(0,0)])
poly.contains(p)
```

```
Out [... True
```

```
In [... poly
```

```
Out [...
```



There are various points that can stand in for the whole object: `centroid` and `representative_point`.

```
In [... poly.centroid
```

```
Out [...
```



```
In [... print(poly.centroid)
```

```
POINT (1 1.5)
```

```
In [... type(poly.centroid)
```

```
Out [... shapely.geometry.point.Point
```

```
In [... rp = poly.representative_point()  
print(rp)
```

```
POINT (1 1.5)
```

```
In [... type(rp)
```

```
Out [... shapely.geometry.point.Point
```

```
In [... q = shapely.Point(4,5)  
q.distance(p)
```

```
Out [... 4.242640687119285
```

A point has an attribute `coords` whose type is:

```
In [... type(p.coords)
```

```
Out [... shapely.coords.CoordinateSequence
```

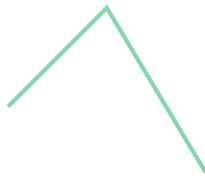
An easy way to get at a `CoordinateSequence` is to turn it into a `list`:

```
In [... list(p.coords)
```

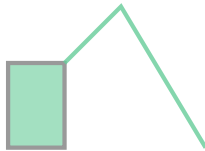
```
Out [... [(1.0, 2.0)]
```

```
In [... r = shapely.Point(7,0)  
LS = shapely.LineString([p,q,r])  
LS
```

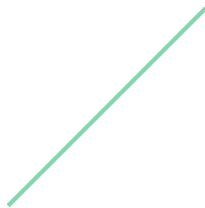
Out [...]

In [...]
`LS.union(poly)`

Out [...]

In [...]
`LS.intersection(poly)`

Out [...]

In [...]
`type(LS)`Out [...]
`shapely.geometry.linestring.LineString`In [...]
`type(LS.coords)`Out [...]
`shapely.coords.CoordinateSequence`In [...]
`list(LS.coords)`Out [...]
`[(1.0, 2.0), (4.0, 5.0), (7.0, 0.0)]`

There is a much more complicated way to dig down into a `CoordinateSequence`:

In [...]
`LS.coords[:][0]`Out [...]
`(1.0, 2.0)`

The brackets colon notation in Python means to make a copy of a list. Here it converts a `CoordinateSequence` object into a

```
list.
```

```
In [... type(LS.coords[:])
```

```
Out [... list
```

Of course, a list can have more than one element. Hence the `[0]`.

```
In [... print(poly)
```

```
POLYGON ((0 0, 2 0, 2 3, 0 3, 0 0))
```

Notice the double parentheses. The reason for this is that it is allowed for a polygon to have holes. (For example, a state might contain lakes).

```
In [... border = [(-180, 90), (-180, -90), (180, -90), (180, 90)]
holes = [(-170, 80), (-170, -80), (170, -80), (170, 80)]
frame = shapely.Polygon(shell=border, holes=holes)
print(frame)
```

```
POLYGON ((-180 90, -180 -90, 180 -90, 180 90, -180 90), (-170 80, -170 -80, 170 -80, 170 80, -170 80))
```

```
In [... frame
```

```
Out [...
```



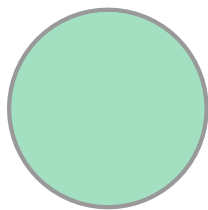
```
In [... print(f"Polygon centroid: {frame.centroid}")
print(f"Polygon Area: {frame.area}")
print(f"Polygon Bounding Box: {frame.bounds}")
```

```
print(f"Polygon Exterior: {frame.exterior}")  
print(f"Polygon Exterior Length: {frame.exterior.length}")
```

Polygon centroid: POINT (0 0)
Polygon Area: 10400.0
Polygon Bounding Box: (-180.0, -90.0, 180.0, 90.0)
Polygon Exterior: LINEARRING (-180 90, -180 -90, 180 -90, 180 90, -180 90)
Polygon Exterior Length: 1080.0

```
In [... # Circle (using a buffer around a point)  
point = shapely.Point((0,0))  
point.buffer(1)
```

Out [...



In [...